

백엔드 개발자 포트폴리오

이종원

문제를 지표로 찾고, 원인을 확인한 뒤 구조로 고치는 백엔드 개발자

2026.03부터 레슨앱의 백엔드와 AI 파이프라인, 입사 전부터 운영되던 무인매장 플랫폼의 백엔드를 혼자 맡고 있음

[GitHub](#) [기술 블로그](#) rnalsakstp2@naver.com

핵심 사례

3.93% → 0%

01 알림 묵음 누락

알림 누락, 같은 부하 3,800건 기준

61.8% → 0%

02 결제 락 경합

락 경합 에러 화면, 결제 콜백 218/353 → 0/60

월 약 \$316

03 하이브리드 STT 파이프라인

STT 비용 절감(월 환산, 장비비 제외), 녹음 549건 중 99.6% Mac mini 처리

13.60% → 9.37%

04 STT 인식 오류율

STT 오류율(CER), 같은 정답지 11구간, 9.37%는 실험값

기술 스택 Java, Spring Boot, JPA, MyBatis, MySQL, Redis, AWS(EC2, RDS, S3, Lambda, SQS), Docker, Python, Prometheus, Grafana

알림 묵음 누락 해결 및 트랜잭션 경계 재설계

2026.07 | 레슨앱 | Spring Boot, JPA, MySQL, 트랜잭션 이벤트, 부하 테스트

배경

레슨앱은 레슨이 끝나면 AI 분석 카드를 만들고 사용자에게 알림을 보냄. 출시 전 부하 감사에서 알림 레코드 일부가 사라짐. 목표는 같은 부하에서 알림 누락 0건.

과제

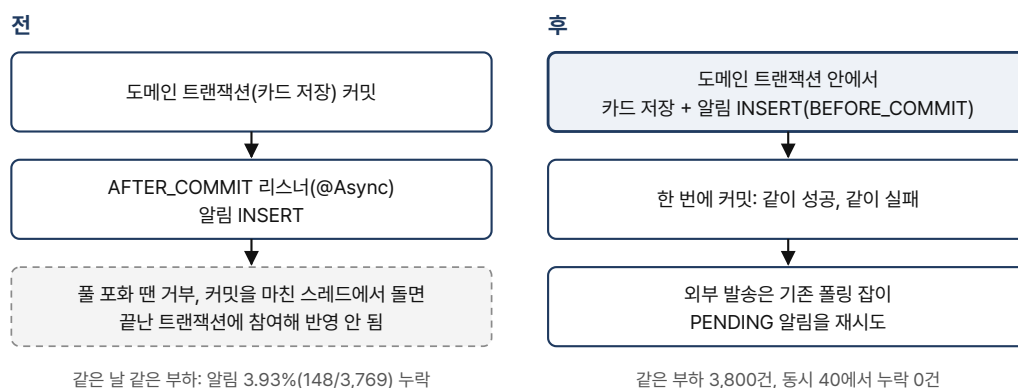
알림 레코드 8.7%(323/3,719, 분모는 생성된 카드 수)가 HTTP 에러 없이 누락되고(로그에는 작업 거부 예외 한 줄뿐) 복구 경로도 없었음. HTTP 지표로는 보이지 않아 어디서 빠지는지부터 찾아야 했음.

행동

- 부하 검증 설계부터 원인 분석, 리스너 재설계, 재측정까지 전담(백엔드 1인)
- Node.js 부하 테스트로 콜백 API 요청 수와 카드, 알림 row 수를 단계별로 대조해 누락 구간을 특정함
- 처음 8.7%는 공유 비동기 풀이 포화되면 기본 거부 정책이 알림 작업을 버린 경로가 주였음. 커밋을 마친 스레드가 직접 실행하게 한 1차 수정은 누락 53%, REQUIRES_NEW를 붙인 2차 수정은 기본 커넥션 풀 10개에서 커넥션 수요가 두 배가 돼 누락 33%와 HTTP 에러 59%(커넥션 풀 고갈 상황)로 실패해 되돌림
- 처음부터 REQUIRES_NEW가 붙은 연습 항목 리스너만 같은 이벤트에서 누락 0이었고, 이 대조군과 Spring 문서의 동작을 종합해, 커밋을 마친 스레드의 알림 INSERT가 끝난 트랜잭션에 참여해 사라지는 경로를 53%의 원인으로, 알림 저장을 커밋 뒤로 미룬 트랜잭션 경계를 근본 원인으로 판단함
- 스레드 풀 튜닝, CDC 릴레이, 별도 outbox 테이블 대신, 리스너 6개에서 @Async를 빼고 BEFORE_COMMIT으로 바꿔 도메인 데이터와 같은 트랜잭션으로 묶음. 외부 발송은 기존 폴링 경로에 그대로 둬

```
@TransactionalEventListener(phase = TransactionPhase.BEFORE_COMMIT)
public void handle(CardCreatedEvent event) {
    notificationRepository.save(Notification.pending(event.userId(), event.cardId()));
}
```

예시 코드: 구조를 보여 주려고 다시 쓴 코드



알림 저장 흐름의 수정 전후

성과

지표	전	후	측정 조건
알림 누락	3.93%(148/3,769)	0%(0/3,800)	같은 날 같은 부하(3,800건, 동시 40), 로컬 단일 A/B 실행
처리량	기준	20% 향상(59.1 → 71.2 TPS)	같은 조건
p50 응답 시간	기준	19% 단축(578 → 470ms)	같은 조건

참고: 8.7%는 동시성 구성이 다른 최초 감사 값이라, 효과는 같은 날 다시 켜 3.93%와만 비교함. 처리량과 응답 시간은 단일 실행 값이며, 비동기 큐 경쟁이 사라진 영향으로 봄

배운 점: HTTP 지표가 정상이어도 데이터는 사라질 수 있음. 요청 수와 저장된 row 수를 대조하는 절차를 이후 크레딧 감사에도 고정 단계로 씀

관련 글: [HTTP 에러 0%, 알림레코드 8.7% 유실](#)

결제 이중 처리 락 경합 해결 및 웹훅 단일 Writer 재설계

2026.04 ~ 2026.06 | 입사 전부터 운영되던 무인매장 플랫폼 | Spring, MyBatis, Redis SET NX 락, PG 웹훅

배경

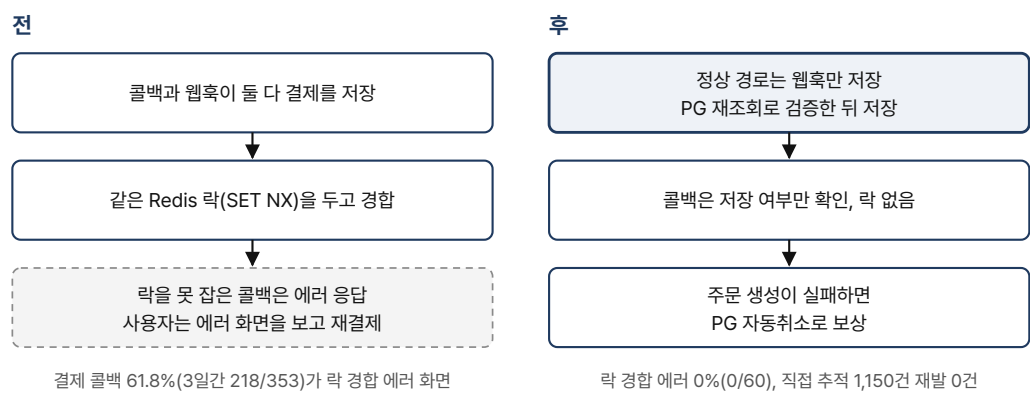
무인매장 플랫폼에서 결제 직후 창을 닫으면 주문이 생기지 않는 CS를 막으려고, 결제 콜백과 웹훅을 모두 저장 경로로 두고 Redis 락으로 중복 처리를 막는 이중 안전망을 직접 넣었음. 목표는 결제가 정상이면 사용자도 정상 화면을 보게 하는 것.

과제

PG에서는 결제가 정상 처리됐는데도 3일간 결제 콜백의 61.8%(218/353)가 락 경합으로 에러 화면을 받아 재결제가 생김. 예외 로그는 0건이라 원인이 보이지 않았음.

행동

- 안전망 설계, 원인 진단, 재설계, 사후 검증 전담(백엔드 1인)
- 예외가 없다는 점에서 정상 흐름 안의 에러 응답을 의심했고, 남은 로그 한 줄로 처음 발견한 날 결제 콜백 85건 중 락 획득 실패 20건이 하나도 복구되지 못하고 모두 에러 화면으로 끝났음을 확인함(3일 구간보다 낮은 비율은 주말 트래픽 차이)
- 락 실패 때 폴링으로 기다리게 하는 안은 버림. 증상은 사라지지만 두 경로가 같은 데이터를 쓰는 구조가 남기 때문. PG 권장 방향으로 정상 경로에서는 웹훅만 저장하고 콜백은 저장 여부만 확인하게 책임을 나눔
- 웹훅은 PG 재조회로 결제를 검증한 뒤 저장하고, 주문 생성 실패 시 PG 자동취소로 보상함. 폴링 코드에 처음 붙였던 읽기 전용 트랜잭션이 5초 내내 커넥션을 붙잡게 된다는 점을 코드 리뷰에서 찾아 제거함
- 남은 한계: 웹훅이 서버에 닿지 못한 결제는 아직 받아낼 장치가 없어, PG 웹훅 재발송 설정과 결제 대사 배치를 후속 과제로 둠



결제 저장 흐름의 수정 전후

성과

지표	전	후	측정 조건
락 경합 에러 화면	61.8%(결제 콜백 218/353)	0%(0/60)	전: 배포 전 3일 운영 기록(락 경합 224건 중 정상 추정 6건 제외), 후: 재설계 뒤 확인한 60건
락 경합	224건(배포 전 3일)	0건(재설계 뒤 60건)	정상 경로의 콜백이 락을 잡지 않는 구조
재발	-	0건	2개월 약 4,500건 중 직접 추적한 1,150건

배운 점: 증상을 덮는 대신 정상 경로에서 저장하는 쪽을 웹훅 하나로 줄이고, 고친 뒤 2개월간 재발을 추적해 확인하는 것까지를 해결로 봄

관련 글: [콜백과 웹훅이 같은 락을 잡으면 생기는 일](#)

Mac mini와 AWS SQS를 활용한 하이브리드 STT 파이프라인 구축

2026.07 ~ 2026.09 | 레슨앱 | AWS Lambda, SQS, Transcribe, whisper.cpp, Mac mini

배경

건별 비용을 관측해 보니(2026.05~06, 60초 이상 녹음 100건) AI 분석 원가의 약 80%가 클라우드 STT 비용이었음. 비용은 줄이되 60분 녹음을 10분 안에 처리한다는 목표는 지켜야 했음.

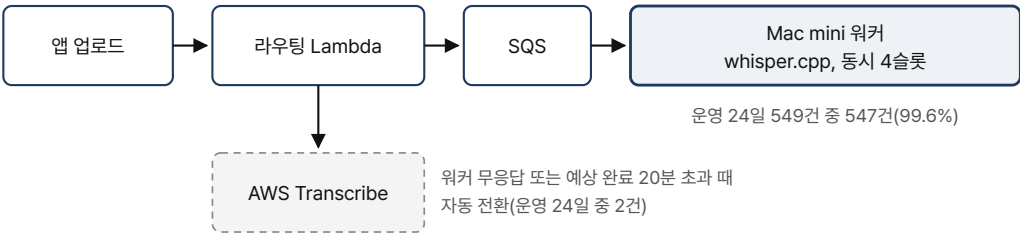
과제

Mac mini 한 대로 전부 처리하면 부하 시뮬레이션에서 10분 목표를 보장할 수 없었음. 비용과 처리 시간을 함께 만족하는 구조가 필요했음.

행동

- 하이브리드 구조 설계, Mac mini 워커 구현, 용량 검증, 운영 전담(백엔드 1인)
- SQS로 받은 작업을 Mac mini whisper.cpp 워커(동시 4슬롯)가 처리하고, 라우팅 Lambda가 워커 무응답이나 예상 완료 시간 20분 초과를 감지하면 Transcribe로 자동 전환하게 함. 10분은 60분 녹음의 처리 시간 목표이고, 20분은 대기열까지 포함한 예상 완료 시간의 상한임
- 전부 로컬로 처리하는 안은 버림. 시뮬레이션에서 10분 목표를 지키는 동시 처리량이 0건이었기 때문. 무거운 파일부터 Transcribe로 넘기는 방식으로 같은 시뮬레이션(실측 28건의 처리 시간으로 계산, Mac mini 한 대에 한꺼번에 몰리는 최악 가정)에서 동시 18건까지 목표를 지킴
- 근거 없이 쓰던 처리량 숫자를 실측 처리 속도와 슬롯 수로 다시 계산해 바로잡음

평소에는 Mac mini, 장애와 과부하 때는 Transcribe



하이브리드 STT 처리 흐름

성과

지표	값	측정 조건
Mac mini 처리 비율	99.6%(547/549)	운영 24일, Transcribe 전환 2건
STT 비용 절감	약 \$253(월 환산 약 \$316)	같은 기간을 Transcribe로 처리했을 때와 비교, 장비비 제외
60분 녹음 처리 시간	중앙값 6.8분(최대 12.8분)	운영 1주(9/13~9/19), 20분 이상 녹음 75건을 60분 기준으로 환산
10분 초과	0건(0/50)	하루 50건을 처리한 날(9/14), 실제 처리 시간 기준

배운 점: 평소 경로와 넘칠 때의 경로를 나누면 비용을 줄이면서 처리 시간 목표도 함께 관리할 수 있음

관련 글: [이벤트 기반 서버리스 AI 추론 파이프라인](#) (전환 전 구조와 비용 관측)

STT 인식 오류율 개선 및 정답지 기반 평가 체계 구축

2026.07 ~ 2026.09 | 레슨앱 | whisper.cpp, Python, CER, WER 채점

배경

레슨 녹음에는 소음과 음악이 섞여 클라우드 STT 오류율이 높았고, 품질을 썰 기준도 없었음. 목표는 개선 여부를 숫자로 판정하는 평가 체계를 만들고 그 기준으로 오류율을 낮추는 것.

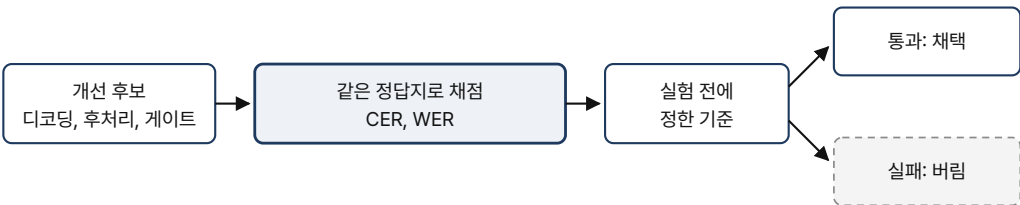
과제

GPU를 함께 쓰는 동시 실행에서는 같은 구간도 결과가 흔들렸고(같은 3분 구간을 3개 동시에 돌렸을 때 텍스트 14~16% 차이), 확신도 같은 간접 지표로는 실제 정확도를 알 수 없었음.

행동

- 평가 체계 설계, 오류 원인 분석, 디코딩, 후처리, 라우팅 개선 전담(백엔드 1인). 실험 코드와 반복 측정은 AI 코딩 도구에 맡기고, 정답지를 만든 뒤로는 채택을 정답지 점수로만 판정함
- 사람이 직접 고친 정답지와 채점기를 만들고, 채택 기준을 실험 전에 정함. 정답지는 실제 레슨이 아닌 8구간(강의 영상 녹음 7구간, 내부 계정 1구간)을 빼고 새 녹음 2구간을 더해 11구간으로 고정함. 개선 전후는 둘 다 이 11구간으로 채점함. 정답지 대부분이 whisper 초안을 고친 것이라 whisper에 유리한 편향이 있음을 감사로 확인해 한계로 기록함
- 잡음 제거는 버림. 초기 실험에서는 확신도가 올랐지만, 바꾸기 전 정답지 17구간(9/8)으로 다시 재니 모두 나빠졌기 때문
- 음악 녹음의 환각 루프는 디코딩 옵션으로 풀어 한 녹음의 10분 구간 전사가 12자에서 1,345자로 돌아옴. VAD가 발화를 통째로 버린 구간은 그 구간만 VAD 없이 다시 전사해 복구함(실제 녹음 7건에서 사람이 확인). 음악 크기를 재는 게이트는 경로가 바뀐 2구간의 CER을 약 21%에서 13~14%로 낮췄고(실험), 9/15부터 운영에 적용함
- GPU 읽기, 쓰기 락으로 동시 실행 때 결과가 흔들리던 문제를 줄임. 락을 켜 뒤 당시 정답지 17구간을 4개씩 동시에 다시 돌려도 결과가 같았음(점수 차이 0.00)

정답지를 만든 뒤에는 점수로만 채택



예: 잡음 제거는 초기에 확신도가 올랐지만 당시 정답지 17구간으로 다시 재니 모두 나빠져 버림
정답지는 사람이 고친 전사본(whisper에 유리한 편향 있음), 9.37%는 게이트를 켜 9/15 설정의 실험값

정답지 기반 채택 흐름

성과

지표	전	후	측정 조건
CER	13.60%	9.37%	같은 정답지 11구간 실험, 전은 첫 구현(7/24), 후는 게이트를 켜 9/15 설정
WER	23.23%	12.65%	같은 조건
게이트 적용 전 CER, WER	-	10.69%, 14.87%	같은 11구간, 9/15 게이트 적용 직전 설정
참고: Transcribe CER	-	18.25%	같은 11구간(정답지가 whisper에 유리)

배운 점: 좋아 보이는 간접 지표보다, 사람이 고친 정답지와 미리 정한 기준이 채택을 결정해야 함

일하는 방식

- **AI를 검증 체계로 씀.** 구현 에이전트와 부하 검증 에이전트의 맥락을 나눠 교차 검증했고, 대조 기준을 직접 설계한 이 감사에서 HTTP 지표로는 보이지 않던 알림 누락을 찾음. QA 에이전트를 직접 만들어 에이전트 7개로 API 154개를 점검했고, 권한 누락(심각도 높음)을 찾아 고친 뒤 다시 검증함(비참여자 403, 없는 레슨 404). 에이전트는 코드를 고치지 않고 보고만 하며, 무엇을 고칠지는 직접 판단함
- **앱 개발자와 API 계약을 맞춤.** 슬랙으로 계약 변경과 연동 오류를 조율함. v3 전환 때 구버전 전용 API 6개를 정리하고, 폐기한 API의 응답을 410으로 통일했으며, Swagger 문서를 전수 조사함
- **운영은 지표로 봄.** Prometheus, Grafana, Loki로 지표와 로그를 모으고, Slack 알림과 GitHub Actions 배포를 직접 구성함

기술 스택: Java, Spring Boot, JPA, MyBatis, MySQL, Redis, AWS(EC2, RDS, S3, Lambda, SQS), Docker, Python, Prometheus, Grafana

연락처: rnalsakstp2@naver.com | github.com/LJW22222 | ljwstudy.tistory.com